

Assignment 4

Implement the following in Python (any other language is also acceptable). Your programs should be interactive or driven by clearly labelled test cases, and should print intermediate steps, not just the final answer.

Task A — Hamming Code Encoder & Decoder

Write a program that:

- Accepts a binary data string of arbitrary length m from the user.
- Computes r using $2^r \geq m + r + 1$ and constructs the $n = m + r$ bit SEC Hamming codeword (even parity), clearly showing which positions are parity bits and which are data bits.
- Simulates transmission by optionally flipping one bit at a position chosen by the user (to emulate a transmission error).
- At the "receiver" side, recomputes the parity checks, prints the syndrome, reports whether an error was detected, and — if a single-bit error is present — corrects it and recovers the original data bits.

Suggested function signatures:

```
def hamming_encode(data_bits: str) -> str: ...
def hamming_decode(codeword: str) -> tuple[str, int, str]:
    # returns (corrected_codeword, syndrome, recovered_data)
```

Task B — CRC Generator & Checker

Write a program that:

- Accepts a binary message and a generator polynomial (as a bit string, e.g. "10011" for $x^4 + x + 1$) from the user.
- Performs modulo-2 division to compute the CRC remainder and produces the transmitted frame (message + remainder).
- Provides a "receiver" routine that takes a (possibly corrupted) frame and the same generator, re-divides, and reports ACCEPT (remainder = 0) or REJECT (remainder $\neq$ 0).
- Lets the user flip an arbitrary bit of the transmitted frame to simulate a transmission error, and shows that the checker correctly rejects it.

Suggested function signatures:

```
def crc_encode(message_bits: str, generator_bits: str) -> tuple[str, str]:
    # returns (transmitted_frame, remainder)
def crc_check(received_bits: str, generator_bits: str) -> tuple[str, bool]:
    # returns (remainder, is_accepted)
```

Expected Sample Run (for self-checking)

Your programs should reproduce results consistent with Exercises H1/H2 and C1/C2 above, e.g.:

```
Data: 11001010
Encoded Hamming codeword (12 bits): 001110001010

Received (with induced error): 001110101010
Syndrome = 7 -> error at position 7
Corrected codeword: 001110001010
Recovered data: 11001010

Message: 1010111 Generator: 1001 ( $x^3 + 1$ )
CRC remainder: 100 Transmitted frame: 1010111100
Receiver check on clean frame -> remainder 000 -> ACCEPT
Receiver check on corrupted frame -> remainder 010 -> REJECT
```

6. Possible Viva-Voce Questions

1. If two valid Hamming codewords differ in exactly 3 bit positions, and a codeword suffers 2 bit errors, will the SEC Hamming decoder always detect that something is wrong? Will it always correct it correctly? Explain.
2. What is the maximum number of data bits that can be protected by an SEC Hamming code using 4 parity bits? Using 5 parity bits?
3. Why does modulo-2 (XOR) division use no borrow or carry, unlike ordinary binary division?
4. Give an example of an error pattern that CRC with a given generator polynomial $G(x)$ would fail to detect. Under what condition does this happen?
5. Compare the redundancy overhead (extra bits per data bits) of Hamming code and CRC-32 for a 1000-bit data frame. Which is more bit-efficient, and why is CRC still preferred on most networks despite not correcting errors?
6. Both hardware (shift-register/XOR circuits) and software can compute a CRC. Briefly describe how a CRC could be computed using a linear feedback shift register (LFSR), as mentioned by Tanenbaum.